

Програмування мовою C++. Вказівники. Оброблення двовимірних масивів. Динамічний розподіл пам'яті.

1. ВКАЗІВНИКИ В C++

Вказівник — це змінна, що містить адресу комірки пам'яті, де зберігається певне значення.

1.1. Оператор адреси &

Повертає адресу змінної.

```
int a = 7;  
cout << &a; // виведе адресу a
```

1.2. Оператор розіменування *

Дозволяє отримати значення за адресою, на яку вказує вказівник.

```
int a = 7;  
cout << *&a; // 7
```

1.3. Оголошення вказівників

```
int *ptr; // ptr — вказівник на int  
double *dptr; // вказівник на double
```

Рекомендація: ставити * біля імені:

```
int* p, q; // p — вказівник, q — НІ  
int *p, *q; // обидва — вказівники
```

1.4. Ініціалізація вказівника

Вказівнику можна присвоювати лише адреси:

```
int value = 5;  
int *ptr = &value;
```

Неправильно:

```
int *ptr = 7; // 7 — не адреса
```

1.5. Узгодження типів

Вказівник повинен відповідати типу змінної:

```
int *p = &x;    // ок  
double *pd = &d; // ок  
int *p2 = &d;   // помилка
```

1.6. Розіменування вказівника

Отримання або присвоєння значення:

```
int a = 5;  
int *p = &a;  
  
*p = 7;    // змінює a  
cout << a; // 7
```

1.7. Некоректні вказівники

- неініціалізований вказівник містить сміття
- спроба розіменування → краш програми

```
int *p;  
cout << *p; // небезпечно!
```

1.8. Розмір вказівника

Залежить від архітектури:

- 32-біт — 4 байти

- 64-біт — 8 байтів

Незалежно від типу:

```
cout << sizeof(int*) << endl;  
cout << sizeof(char*) << endl;  
// однаковий розмір
```

1.9. Навіщо потрібні вказівники

робота з масивами
динамічна пам'ять (new, delete)
передача великих структур у функції без копіювання
передача функцій як параметрів
реалізація поліморфізму
створення зв'язних структур (списки, дерева)

2. Рядки типу `std::string`

`std::string` — це клас для роботи з текстом із великою кількістю готових методів.
Підключення:

```
#include <string>
```

2.1. Ініціалізація

```
string s1 = "Hello";  
string s2("World");  
string s3(5, 'a'); // "aaaaa"
```

2.2. Конкатенація

```
string a = "Hello";  
string b = " World";  
string c = a + b;
```

2.3. Доступ до символів

```
char x = s[2]; // без перевірки  
char y = s.at(2); // з перевіркою
```

2.4. Додавання

```
s.push_back('!');  
s.append(" C++");
```

2.5. Розмір

```
s.length();  
s.size();
```

2.6. Перевірка порожнечі

```
s.empty();  
s.clear();
```

2.7. Порівняння

```
if (s1.compare(s2) == 0) // рівні
```

2.8. Пошук

```
size_t pos = s.find("World");  
if (pos != string::npos) ...
```

2.9. Підрядок

```
string sub = s.substr(0, 5);
```

2.10. Вставка та видалення

```
s.insert(5, ",");  
s.erase(5, 1);
```

2.11. Заміна

```
s.replace(6, 5, "C++");
```

2.12. Перетворення

Рядок → число:

```
stoi("123");    // 123  
stod("12.5");   // 12.5
```

Число → рядок:

```
to_string(45);  // "45"
```

3. Порівняння C-рядків та std::string

Функція	C-рядки (char[])	std::string
довжина	strlen()	.size()
копіювання	strcpy()	=
конкатенація	strcat()	+, .append()
порівняння	strcmp()	.compare(), ==
пошук	strchr(), strstr()	.find()
тип	сирий масив	клас із методами
безпека	низька	висока